

System on Programmable Chip Design

Lecture: Avalon Interface, part 2

Lecturer: Dr. Nguyen Anh Vinh

Ref: Truong Quang Vinh, IT-SoC Lab

❖ Class content

- Avalon-MM interface
- Avalon-MM tristate interface
- Avalon-ST interface
- Conduit interface

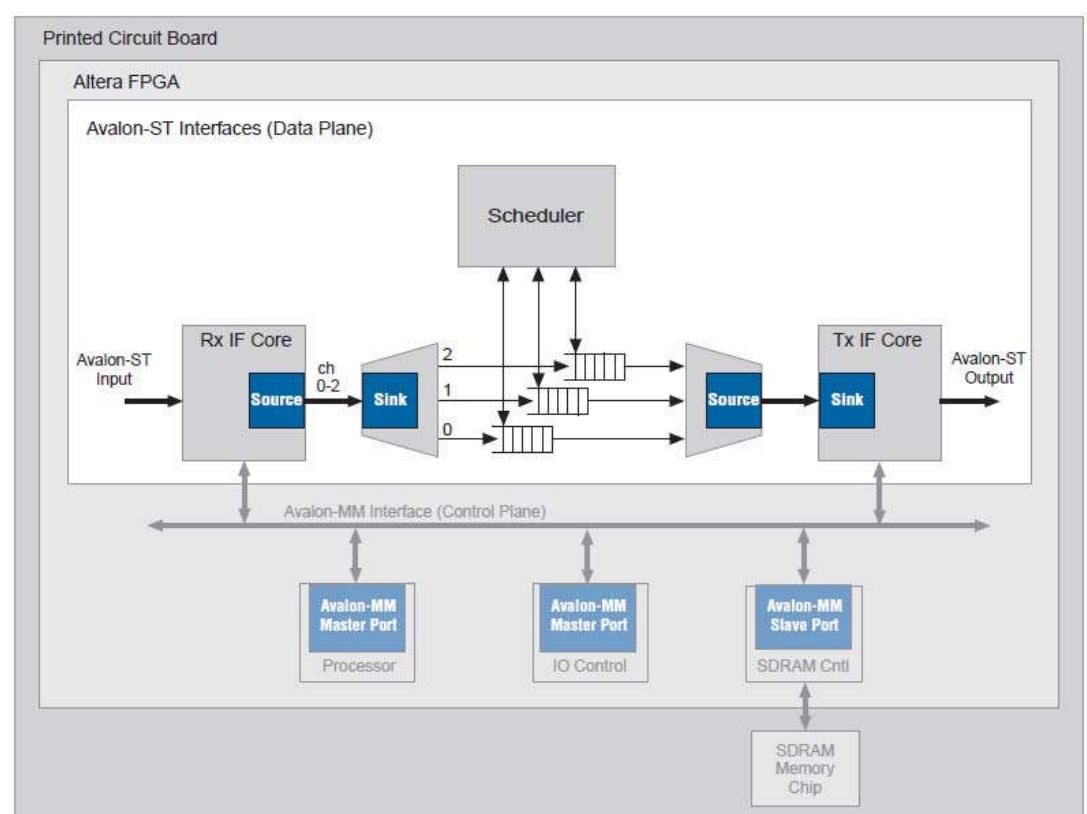
❖ Required documents

- Avalon interface specification
(http://www.altera.com/literature/manual/mnl_avalon_spec.pdf)

1. **Avalon Memory Mapped Interface (Avalon-MM)**—an address-based read/write interface typical of master-slave connections.
2. **Avalon Streaming Interface (Avalon-ST)**—an interface that supports the unidirectional flow of data, including multiplexed streams, packets, and DSP data.
3. **Avalon Memory Mapped Tristate Interface**—an address-based read/write interface to support off-chip peripherals.
4. **Avalon Clock**—an interface that drives or receives clock and reset signals to synchronize interfaces and provide reset connectivity.
5. **Avalon Interrupt**—an interface that allows components to signal events to other components.
6. **Avalon Conduit**—an interface that allows signals to be exported out at the top level of an SOPC Builder system

❖ Avalon-ST interface:

- High bandwidth
- Low latency
- Unidirectional data
- Multiple channel
- Burst mode



❖ *Avalon Streaming System*

- is a system that contains one or more Avalon-ST connections that transfer data from a source interface to a sink interface.

❖ *Avalon Streaming Components*

- A typical system using Avalon-ST interfaces combines multiple functional modules, called components. The system designer configures the components and connects them together to implement a system.

❖ *Source and Sink Interfaces and Connections*

- When two components are connected, the data flows from the source interface to the sink interface.
- The combination of a source interface connected to a sink interface is referred to as a connection.

❖ *Backpressure*

- is a mechanism by which a sink can signal to a source to stop sending data.
- The sink typically uses backpressure to stop the flow of data when its FIFOs are full or when there is congestion on its output port.

❖ *Transfers and Ready Cycles*

- A transfer is an operation that results in data and control propagation from a source interface to a sink interface.
- A ready cycle is a cycle during which the sink can accept a transfer.

❖ *Symbol*

- A symbol is the smallest unit of data.
- For most packet interfaces, a symbol is a byte.
- One or more symbols make up the single unit of data transferred in a cycle.

❖ *Channel*

- A channel is a physical or logical path or link through which information passes between two ports.

❖ *Packet*

- A packet is an aggregation of data and control signals that is transmitted together.
- A packet may contain a header to help routers and other network devices direct the packet to the correct destination.
- The packet format is defined by the application, not this specification.
- Avalon-ST packets can be variable in length and can be interleaved across a connection.

Table 6–1. Avalon-ST Interface Signals

Signal Type	Width	Direction	Required	Description
Fundamental Signals				
ready	1	Sink → Source	No	Asserted high to indicate that the sink can accept data. On interfaces supporting flow control, <code>ready</code> is asserted by the sink on cycle $\langle n \rangle$ to mark cycle $\langle n + \text{readyLatency} \rangle$ as a ready cycle, during which the source may assert <code>valid</code> and transfer data. Sources without a <code>ready</code> input cannot be backpressured, and sinks without a <code>ready</code> output never need to backpressure.
valid	1	Source → Sink	No	Asserted by the source to qualify all other source to sink signals. On ready cycles where <code>valid</code> is asserted, the data bus and other source to sink signals are sampled by the sink, and on other cycles are ignored. Sources without a <code>valid</code> output implicitly provide valid data on every cycle that they are not being backpressured, and sinks without a <code>valid</code> input expect valid data on every cycle that they are not backpressuring.
data	1–256	Source → Sink	No	The <code>data</code> signal from the source to the sink, typically carries the bulk of the information being transferred. The contents and format of the <code>data</code> signal is further defined by parameters.

channel	0–127	Source → Sink	No	The <code>channel</code> number for data being transferred on the current cycle. If an interface supports the <code>channel</code> signal, it must also define the <code>maxChannel</code> parameter.
error	1–255	Source → Sink	No	A bit mask used to mark errors affecting the data being transferred in the current cycle. A single bit in <code>error</code> is used for each of the errors recognized by the component, as defined by the <code>errorDescriptor</code> property.
Packet Transfer Signals				
startofpacket	1	Source → Sink	No	Asserted by the source to mark the beginning of a packet.
endofpacket	1	Source → Sink	No	Asserted by the source to mark the end of a packet.
empty	0–8	Source → Sink	No	Indicates the number of symbols that are empty during cycles that contain the end of a packet. The <code>empty</code> signal is not used on interfaces where there is one symbol per beat. If <code>endofpacket</code> is not asserted, this signal is not interpreted.

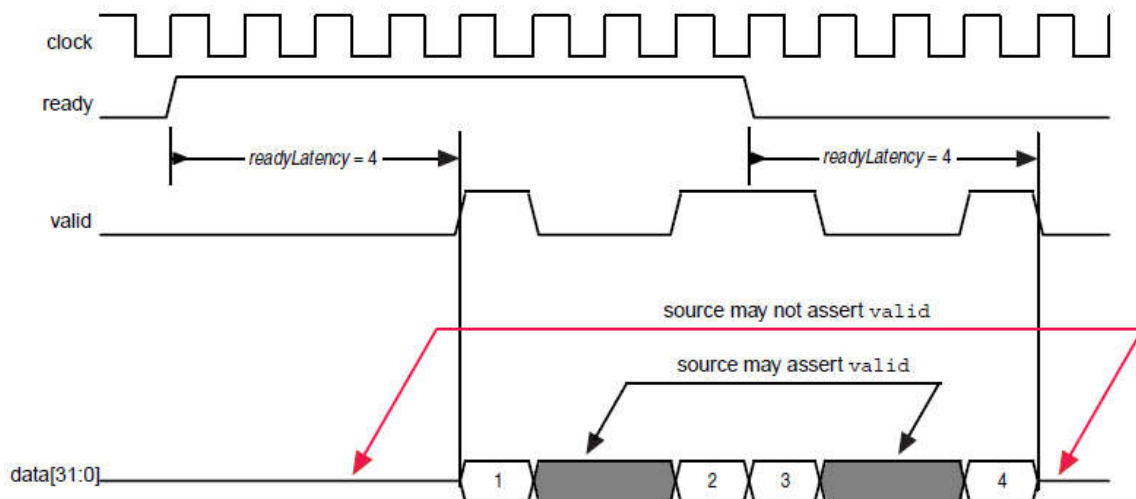
Table 6–2. Avalon-ST Interface Properties

Property Name	Default Value	Legal Values	Description
dataBitsPerSymbol	8	1–512	Defines the number of bits per symbol. For example, byte-oriented interfaces have 8-bit symbols. This value is not restricted to be a power of 2.
readyLatency	0	0–8	Defines the relationship between assertion/deassertion of the ready signal, and cycles which are considered to be <i>ready</i> for data transfer, separately for each interface.
maxChannel	0	0–255	The maximum number of channels that a data interface can support.
errorDescriptor	0	list of strings	A list of words that describe the error associated with each bit of the error signal. The length of the list must be the same as the number of bits in the error signal, and the first word in the list applies to the highest order bit. For example, “ <i>crc, overflow</i> ” means that bit[1] of <i>error</i> indicates a CRC error, and bit[0] indicates an overflow error.

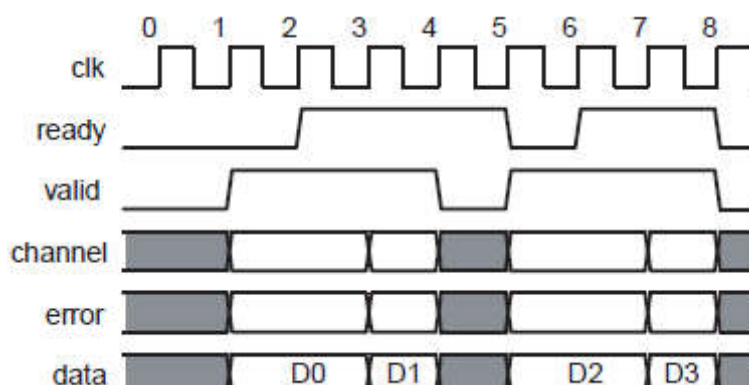
- ❖ On any given clock cycle, the source interface drives the data and the optional channel and error signals, and asserts valid.
- ❖ The sink interface samples these signals on the rising edge of the reference clock if valid is asserted



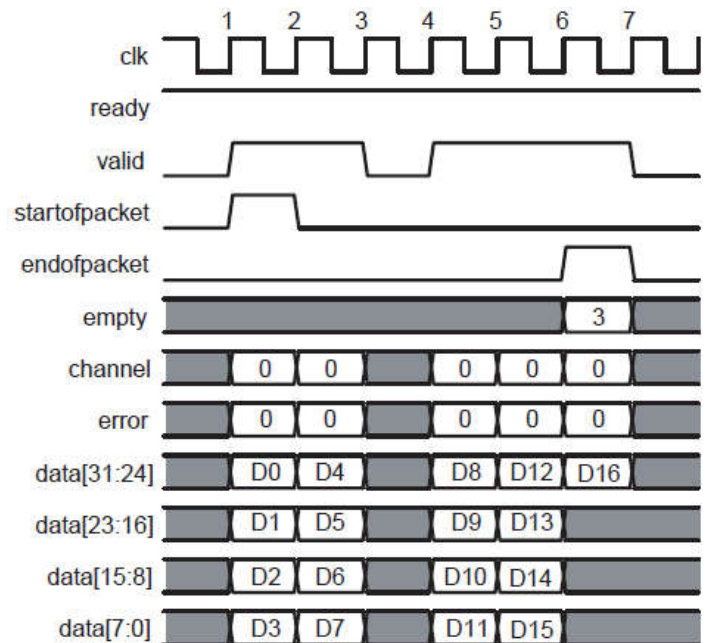
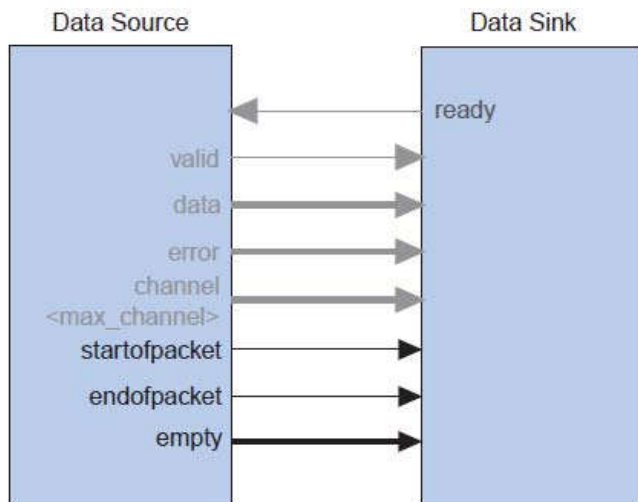
- ❖ The sink indicates to the source that it is ready for an active cycle by asserting ready for a single clock cycle.
- ❖ Cycles during which the sink is ready for data are called ready cycles.
- ❖ During a ready cycle, the source may assert valid and provide data to the sink.
- ❖ If it has no data to send, it deasserts valid and can drive data to any value.



- ❖ The source provides data and asserts valid on cycle 1, even though the sink is not ready.
- ❖ The source waits until cycle two, when the sink does assert ready, before moving onto the next data cycle.
- ❖ In cycle 3, the source drives data on the same cycle and the sink is ready to receive it; the transfer happens immediately.
- ❖ In cycle 4, the sink asserts ready, but the source does not drive valid data.



- ❖ Startofpacket: indicate the cycle that start to transfer a packet
- ❖ Endofpacket: indicate the cycle that end to transfer a packet
- ❖ Empty: indicate the number of symbols that are empty at the end of packet



- ❖ Conduit interfaces
 - Used to export to the outside of the SOPC system
 - Include Input/output signals

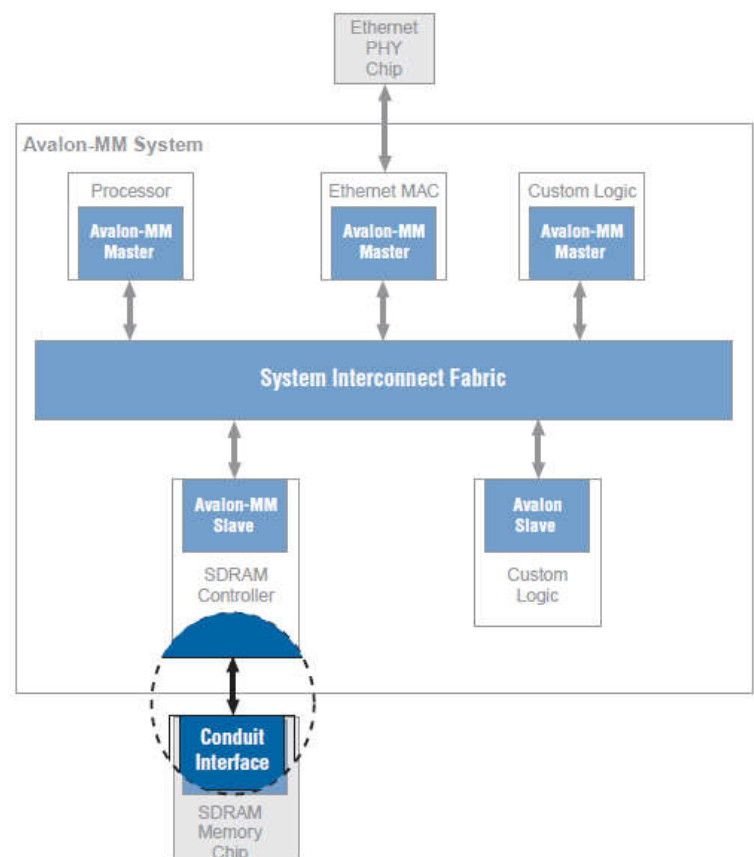


Table 7–1. Conduit Signal Types

Signal Type	Width	Direction	Required	Description
export	n	In, out or bidirectional	Yes	A conduit interface consists of one or more signals of arbitrary width of direction input or output, of type <code>export</code> . All of these signals are exported out the top level of the SOPC Builder system.

❖ Avalon-ST Video protocol:

- To send video and control data over Avalon-ST connections
- Allow to changes in the format of the video being processed

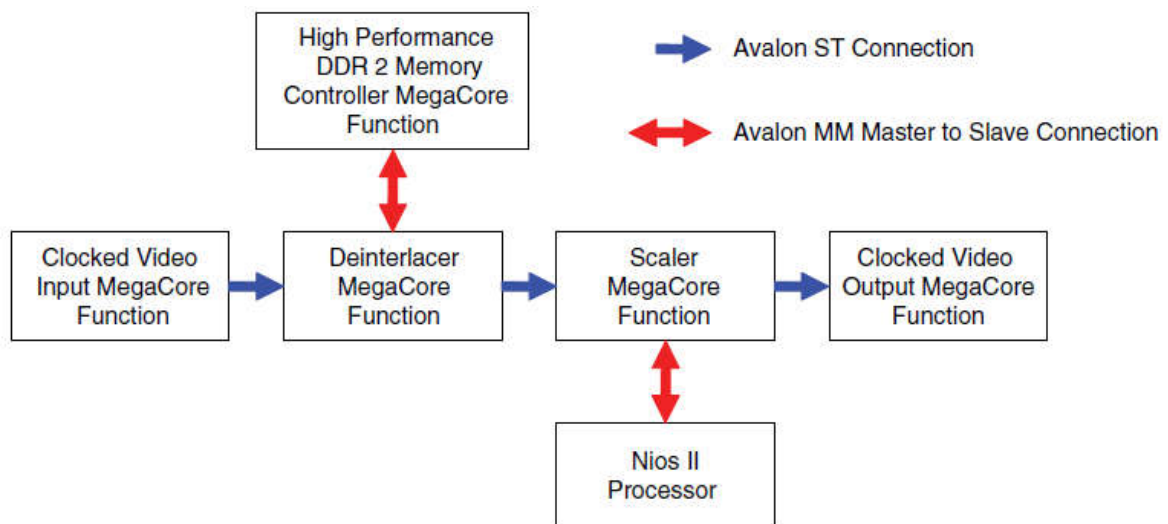


Table 4-6. Avalon-ST Interface Parameters

Parameter Name	Value
BITS_PER_SYMBOL	Variable. Always equal to the Bits per Color Sample parameter value of the stream of pixel data being transferred.
SYMBOLS_PER_BEAT	Variable. Always equal to the number of color samples being transferred in parallel. This is equivalent to the number of rows in the color pattern parameter value of the stream of pixel data being transferred.
READY_LATENCY	1

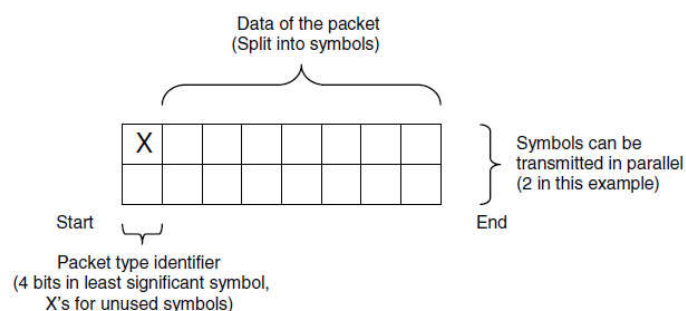
Table 4-7. Avalon-ST Interface Signal Types

Signal	Width	Direction
ready	1	Sink to Source
valid	1	Source to Sink
data	$\text{bits_per_symbol} \times \text{symbols_per_beat}$	Source to Sink
startofpacket	1	Source to Sink
endofpacket	1	Source to Sink

- ❖ Packets: two types of packets
 - Video data packets containing only uncompressed video data
 - Control data packets containing the control data used to configure the cores for incoming video data packets
- ❖ The packet type is defined by a 4-bit packet type identifier

Table 4-1. Avalon-ST Video Packet Types

Type Identifier	Description
0	Video data packet
1–8	User packet types
9–14	Reserved for future Altera use
15	Control data packet



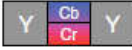
❖ Color patterns:

- Parallel or serial
- RGB or YCbCr
- 4:4:4 or 4:2:2

Table 4–2. Examples of Static Avalon-ST Video Data Packet Parameters

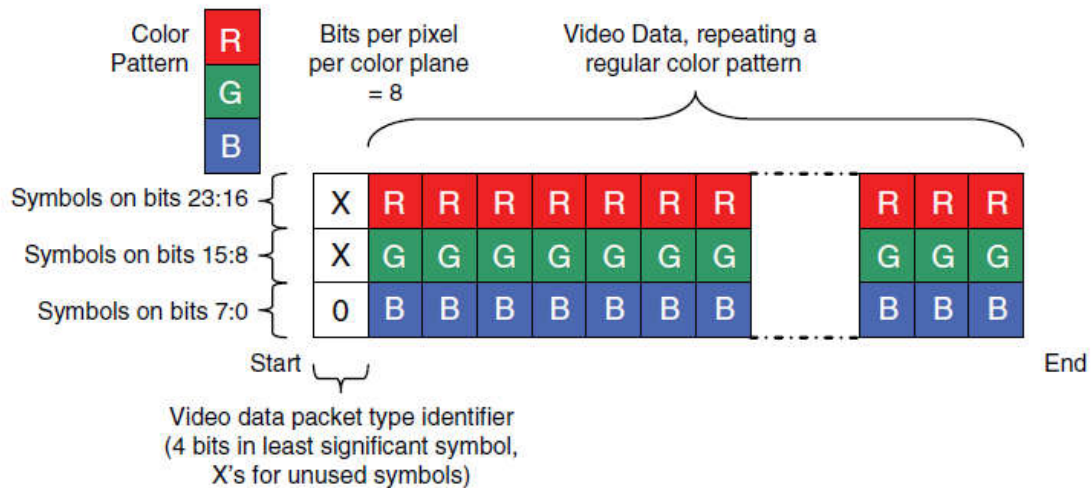
Parameters		Description
Bits per Color Sample	Color Pattern	
8		Three color planes, B', G', and R' are transmitted in alternating sequence and each B', G', or R' sample is represented using 8 bits of data.
10		Three color planes are transmitted in parallel, leading to higher throughput than when transmitted in sequence, usually at higher cost. Each R', G', or B' sample is represented using 10 bits of data, so that, in total, 30 bits of data are transmitted in parallel.
10		4:2:2 video in the Y'CbCr color space, where there are twice as many Y' samples as Cb or Cr samples. One Y' sample and one of either a Cb or a Cr sample is transmitted in parallel. Each sample is represented using 10 bits of data.

Table 4–3. Recommended Color Patterns

Color Space	Recommended Color Patterns	
	Parallel	Sequence
R'G'B'		
Y'CbCr		
4:2:2 Y'CbCr		
4:2:0 Y'CbCr		

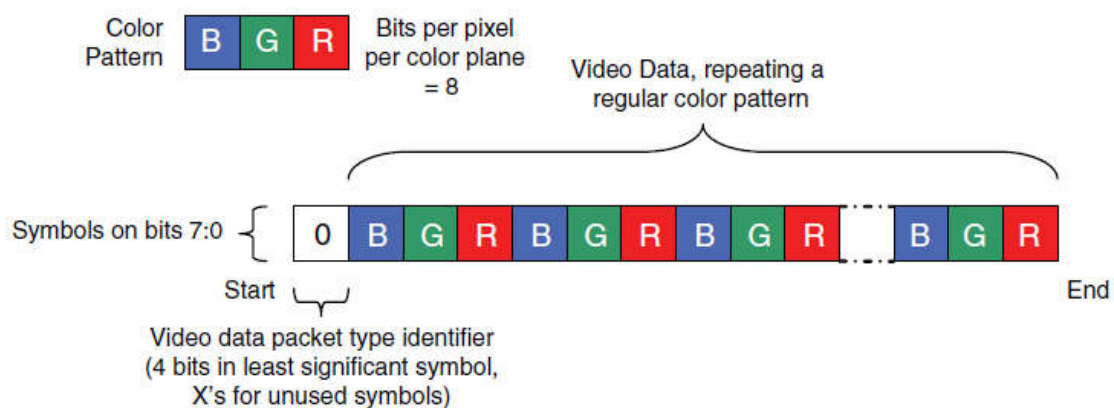
- ❖ The structure of a video data packet using a set parallel color pattern and bits per pixel per color plane

Figure 4-6. Parallel Color Pattern



- ❖ The structure of a video data packet using a set sequential color pattern and bits per pixel per color plane

Figure 4-7. Sequence Color Pattern



- ❖ A control data packet complies with the standard of a packet type identifier followed by a data payload.
- ❖ The data payload is split into nibbles of 4 bits, each data nibble is part of a symbol.
- ❖ If the width of a symbol is greater than 4 bits, the most significant bits of the symbol is not used.

Table 4–5. Order of Nibbles and Associated Symbols

Order	Symbol	Order	Symbol
1	width[15..12]	6	height[11..8]
2	width[11..8]	7	height[7..4]
3	width[7..4]	8	height[3..0]
4	width[3..0]	9	interlacing[3..0]
5	height[15..12]		

Table 4–4. Examples of Control Data Packet Parameters

Type	Parameters			Description
	Width	Height	Interlacing	
15	1920	1080	0011	The frames that follow are progressive with a resolution of 1920×1080.
15	640	480	0011	The frames that follow are progressive with a resolution of 640×480.
15	640	480	0000	The frames that follow are progressive with a resolution of 640×480. The frames were deinterlaced using f0 as the last field.
15	640	480	0001	The frames that follow are progressive with a resolution of 640×480. The frames were deinterlaced using f1 as the last field.
15	640	480	1000	The fields that follow are 640 pixels wide and 240 pixels high. The next field is f0 (even lines) and it is paired with the f1 field that precedes it.
15	1920	540	1100	The fields that follow are 1920 pixels wide and 540 pixels high. The next field is f1 (odd lines) and it is paired with the f0 field that follows it.
15	1920	540	1101	The fields that follow are 1920 pixels wide and 540 pixels high. The next field is f1 (odd lines) and it is paired with the f0 field that precedes it.
15	1920	540	1011	The fields that follow are 1920 pixels wide and 540 pixels high. The next field is f0 (even lines) and the stream should be handled as genuine interlaced video material where the fields are all temporally disjoint.
15	1920	540	1010	The fields that follow are 1920 pixels wide and 540 pixels high. The next field is f0 (even lines) and the stream should be handled as genuine interlaced video content although it may originate from a progressive source converted with a pull-down.

Figure 4–8. Three Symbols in Parallel

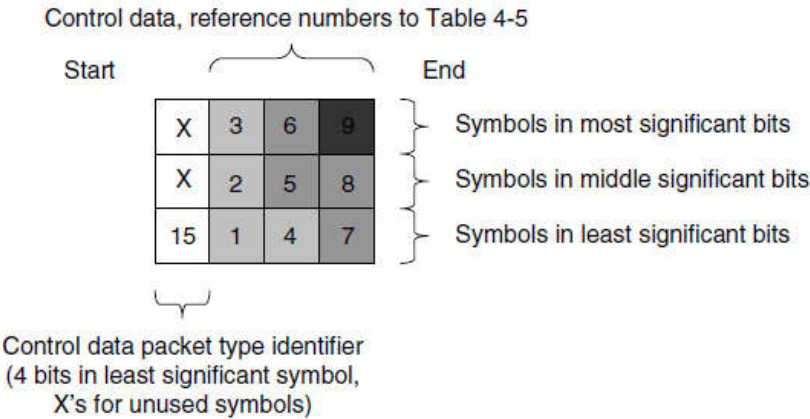


Figure 4–9. Two Symbols in Parallel

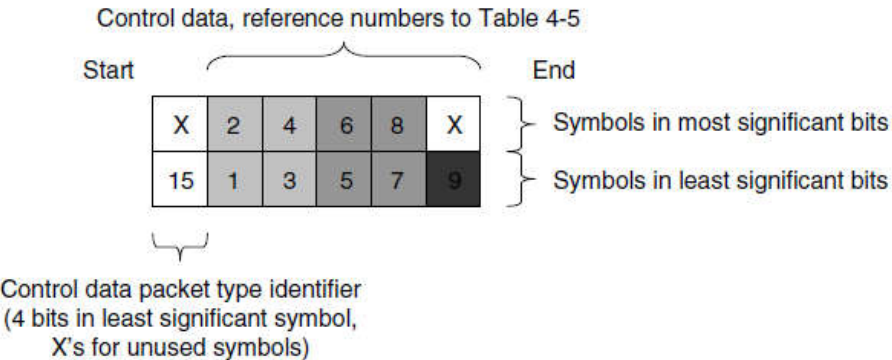


Figure 4–10. One Symbol in Parallel

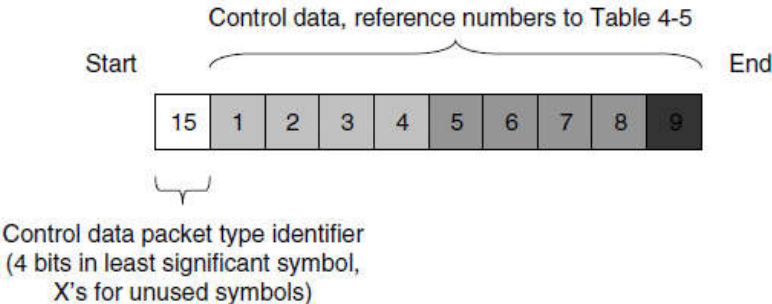


Figure 4–11. Timing Diagram Showing R'G'B' Transferred in Parallel

